

10

Applications of Object Detection and Segmentation

In previous chapters, we learned about various object detection techniques, such as the R-CNN family of algorithms, YOLO, SSD, and the U-Net and Mask R-CNN image segmentation algorithms. In this chapter, we will take our learning a step further – we will work on more realistic scenarios and learn about frameworks/architectures that are more optimized to solve detection and segmentation problems.

We will start by leveraging the Detectron2 framework to train and detect custom objects present in an image. We will also predict the pose of humans present in an image using a pre-trained model. Furthermore, we will learn how to count the number of people in a crowd in an image and then learn about leveraging segmentation techniques to perform image colorization. Next, we will learn about a modified version of YOLO to predict 3D bounding boxes around objects by using point clouds obtained from a LIDAR sensor. Finally, we will learn about recognizing actions from a video.

By the end of this chapter, you will have learned about the following:

- Multi-object instance segmentation
- Human pose detection
- Crowd counting
- Image colorization
- 3D object detection with point clouds
- Action recognition from video



All code snippets within this chapter are available in the `Chapter10` folder of the GitHub repository at <https://bit.ly/mcvc-2e>.

As the field evolves, we will periodically add valuable supplements to the GitHub repository. Do check the `supplementary_sections` folder within each chapter's directory for new and useful content.

Multi-object instance segmentation

In previous chapters, we learned about various object detection algorithms. In this section, we will learn about the Detectron2 platform (<https://ai.facebook.com/blog/-detectron2-a-pytorch-based-modular-object-detection-library-/>) before we implement multi-object instance segmentation using the Google Open Images dataset. Detectron2 is a platform built by the Facebook team. Detectron2 includes high-quality

implementations of state-of-the-art object detection algorithms, including DensePose of the Mask R-CNN model family. The original Detectron framework was written in Caffe2, while the Detectron2 framework is written using PyTorch.

Detectron2 supports a range of tasks related to object detection. Like the original Detectron, it supports object detection with boxes and instance segmentation masks, as well as human pose prediction. Beyond that, Detectron2 adds support for semantic segmentation and panoptic segmentation (a task that combines both semantic and instance segmentation). By leveraging Detectron2, we are able to build object detection, segmentation, and pose estimation in a few lines of code.

In this section, we will:

- Fetch the open-images dataset
- Convert the dataset to COCO format
- Train a model using Detectron2
- Infer new images on the trained model

Let's go through each of these.

Fetching and preparing data

We will be working on the images that are available in the Open Images dataset (which contains millions of images along with their annotations) provided by Google at

<https://storage.googleapis.com/openimages/web/index.html>.

In this part of the code, we will learn about fetching only the required images and not the entire dataset. Note that this step is required, as the dataset size prohibits a typical user who might not have extensive resources from building a model:



The following code can be found in the `Multi_object_segmentation.ipynb` file located in the `Chapter10` folder on GitHub at <https://bit.ly/mcvcv-2e>.

1. Install the required packages:

```
!pip install -qU openimages torch_snippets
```

2. Download the required annotations files:

```
from torch_snippets import *
!wget -O train-annotations-object-segmentation.csv -q https://storage.googleapis.com/openimage:
!wget -O classes.csv -q \
https://raw.githubusercontent.com/openimages/dataset/master/dict.csv
```

3. Specify the classes that we want our model to predict (you can visit the Open Images website to see the list of all classes):

```

required_classes = 'person,dog,bird,car,elephant,football,\
jug,laptop,mushroom,pizza,rocket,shirt,traffic sign,\
Watermelon,Zebra'
required_classes = [c.lower() for c in \
                    required_classes.lower().split(',')]
classes = pd.read_csv('classes.csv', header=None)
classes.columns = ['class', 'class_name']
classes = classes[classes['class_name'].map(lambda x: x \
                                           in required_classes)]

```

4. Fetch the image IDs and masks corresponding to `required_classes`:

```

from torch_snippets import *
df = pd.read_csv('train-annotations-object-segmentation.csv')
data = pd.merge(df, classes, left_on='LabelName',
                right_on='class')
subset_data = data.groupby('class_name').agg( \
    {'ImageID': lambda x: list(x)[:500]})
subset_data = flatten(subset_data.ImageID.tolist())
subset_data = data[data['ImageID'].map(lambda x: x in subset_data)]
subset_masks = subset_data['MaskPath'].tolist()

```

Given the vast amount of data, we are only fetching 500 images per class in `subset_data`. It is up to you whether you fetch a smaller or larger set of files per class and the list of unique classes (`required_classes`).

So far, we only have the `ImageId` and `MaskPath` values corresponding to an image. In the next steps, we will go ahead and download the actual images and masks from `open-images`.

5. Now that we have the subset of masks data to download, let's start the download. Open Images has 16 ZIP files for training masks. Each ZIP file will have only a few masks from `subset_masks`, so we will delete the rest after moving the required masks into a separate folder. This *download -> move -> delete* action will keep the memory footprint relatively small. We will have to run this step once for each of the 16 files:

```

!mkdir -p masks
for c in Tqdm('0123456789abcdef'):
    !wget -q \
        https://storage.googleapis.com/openimages/v5/train-masks/train-masks-{c}.zip
    !unzip -q train-masks-{c}.zip -d tmp_masks
    !rm train-masks-{c}.zip
    tmp_masks = Glob('tmp_masks', silent=True)
    items = [(m, fname(m)) for m in tmp_masks]
    items = [(i, j) for (i, j) in items if j in subset_masks]
    for i, j in items:
        os.rename(i, f'masks/{j}')
    !rm -rf tmp_masks

```

6. Download the images corresponding to `ImageId`:

```

masks = Glob('masks')
masks = [fname(mask) for mask in masks]
subset_data=subset_data[subset_data['MaskPath'].map(lambda \
                                                    x: x in masks)]

```

```
subset_imageIds = subset_data['ImageID'].tolist()
from openimages.download import _download_images_by_id
!mkdir images
_download_images_by_id(subset_imageIds, 'train', './images/')
```

7. Zip all the images, masks, and ground truths and save them – just in case your session crashes, it is helpful to save and retrieve the file for later training. Once the ZIP file is created, ensure you save the file in your drive or download it. The file size ends up being around 2.5 GB:

```
import zipfile
files = Glob('images') + Glob('masks') + \
['train-annotations-object-segmentation.csv', 'classes.csv']
with zipfile.ZipFile('data.zip', 'w') as zipme:
    for file in Tqdm(files):
        zipme.write(file, compress_type=zipfile.ZIP_DEFLATED)
```

Finally, move the data into a single directory:

```
!mkdir -p train/
!mv images train/myData2020
!mv masks train/annotations
```

Given that there are so many moving components in object detection code, as a standardization method, Detectron accepts a rigid data format for training. While it is possible to write a dataset definition and feed it to Detectron, it is easier (and more profitable) to save the entire training data in COCO format. This way, you can leverage other training algorithms, such as **detectron transformers (DETR)**, with no change to the data.

8. First, we will start by defining the categories of classes:

1. Define the required categories in COCO format:

```
!pip install git+git://github.com/sizhky/pycococreator.git@0.2.0
import datetime
INFO = {
    "description": "MyData2020",
    "url": "None",
    "version": "1.0",
    "year": 2020,
    "contributor": "sizhky",
    "date_created": datetime.datetime.utcnow().isoformat(' ')
}
LICENSES = [
    {
        "id": 1,
        "name": "MIT"
    }
]
CATEGORIES = [{ 'id': id+1, 'name': name.replace('/', ''), \
    'supercategory': 'none' } \
    for id, (_, (name, class_name)) in \
    enumerate(classes.iterrows())]
```

In the preceding code, in the definition of `CATEGORIES`, we are creating a new key called `supercategory`. To understand `supercategory`, let's go through an example: the `Man` and `Woman` classes are categories belonging to the `Person` supercategory. In our case, given that we are not interested in supercategories, we will specify it as `none`.

2. Import the relevant packages and create an empty dictionary with the keys needed to save the COCO JSON file:

```
!pip install pycocotools
from pycococreatortools import pycococreatortools
from os import listdir
from os.path import isfile, join
from PIL import Image
coco_output = {
    "info": INFO,
    "licenses": LICENSES,
    "categories": CATEGORIES,
    "images": [],
    "annotations": []
}
```

3. Set a few variables in place that contain the information on the image locations and annotation file locations:

```
ROOT_DIR = "train"
IMAGE_DIR, ANNOTATION_DIR = 'train/myData2020/', 'train/annotations/'
image_files = [f for f in listdir(IMAGE_DIR) if \
                isfile(join(IMAGE_DIR, f))]
annotation_files = [f for f in listdir(ANNOTATION_DIR) if \
                    isfile(join(ANNOTATION_DIR, f))]
```

4. Loop through each image filename and populate the `images` key in the `coco_output` dictionary:

```
image_id = 1
# go through each image
for image_filename in Tqdm(image_files):
    image = Image.open(IMAGE_DIR + '/' + image_filename)
    image_info = pycococreatortools.create_image_info(image_id, \
                                                         os.path.basename(image_filename), image.size)
    coco_output["images"].append(image_info)
    image_id = image_id + 1
```

9. Loop through each segmentation annotation and populate the `annotations` key in the `coco_output` dictionary:

```
segmentation_id = 1
for annotation_filename in Tqdm(annotation_files):
    image_id = [f for f in coco_output['images'] if \
                stem(f['file_name']) == \
                annotation_filename.split('_')[0]][0]['id']
    class_id = [x['id'] for x in CATEGORIES \
                if x['name'] in annotation_filename][0]
    category_info = {'id': class_id, \
                     'is_crowd': 'crowd' in image_filename}
```

```

binary_mask = np.asarray(Image.open(f'{ANNOTATION_DIR}/{
{annotation_filename}').convert('1')).astype(np.uint8)

annotation_info = pycococreatortools\
    .create_annotation_info( \
        segmentation_id, image_id,
        category_info,
        binary_mask, image.size, tolerance=2)
if annotation_info is not None:
    coco_output["annotations"].append(annotation_info)
    segmentation_id = segmentation_id + 1

```

10. Save `coco_output` in a JSON file:

```

coco_output['categories'] = [{ 'id': id+1, 'name': cls_name, \
    'supercategory': 'none' } for \
    id, (_, (name, cls_name)) in \
    enumerate(classes.iterrows())]

import json
with open('images.json', 'w') as output_json_file:
    json.dump(coco_output, output_json_file)

```

With this, we have our files in COCO format. Now, we are ready to train our model using the Detectron2 framework.

Training the model for instance segmentation

To train our model, we need to download the required packages, modify the configuration file to reflect the dataset paths, and then train the model.

Let's go through the steps:

1. Install the required Detectron2 packages:

```

%cd /content/
# install detectron2:
!git clone https://github.com/facebookresearch/detectron2
%cd /content/detectron2
%pip install -r requirements.txt
!python setup.py install
%pip install git+https://github.com/facebookresearch/fvcore.git
%cd /content/

```

 Note: You'll need to restart Colab before proceeding to the next step.

2. Import the relevant `detectron2` packages:

```

from detectron2 import model_zoo
from detectron2.engine import DefaultPredictor
from detectron2.config import get_cfg
from detectron2.utils.visualizer import Visualizer
from detectron2.data import MetadataCatalog, DatasetCatalog
from detectron2.engine import DefaultTrainer

```

Given that we have restarted Colab, let's re-fetch the required classes:

```
from torch_snippets import *
required_classes= 'person,dog,bird,car,elephant,football,jug,\
laptop,Mushroom,Pizza,Rocket,Shirt,Traffic sign,\
Watermelon,Zebra'
required_classes = [c.lower() for c in \
                    required_classes.lower().split(',')]
classes = pd.read_csv('classes.csv', header=None)
classes.columns = ['class', 'class_name']
classes = classes[classes['class_name'].map(lambda \
                                         x: x in required_classes)]
```

3. Register the created datasets using `register_coco_instances`:

```
from detectron2.data.datasets import register_coco_instances
register_coco_instances("dataset_train", {}, \
                      "images.json", "train/myData2020")
```

4. Define all the parameters in the `cfg` configuration file. Configuration (`cfg`) is a special Detectron object that holds all the relevant information for training a model:

```
cfg = get_cfg()
cfg.merge_from_file(model_zoo.get_config_file("COCO- InstanceSegmentation/mask_rcnn_R_50_FPN_
cfg.DATASETS.TRAIN = ("dataset_train",)
cfg.DATASETS.TEST = ()
cfg.DATALOADER.NUM_WORKERS = 2
cfg.MODEL.WEIGHTS = model_zoo.get_checkpoint_url("COCO- InstanceSegmentation/mask_rcnn_R_50_F
# weights
cfg.SOLVER.IMS_PER_BATCH = 2
cfg.SOLVER.BASE_LR = 0.00025 # pick a good LR
cfg.SOLVER.MAX_ITER = 5000 # instead of epochs, we train on
# 5000 batches
cfg.MODEL.ROI_HEADS.BATCH_SIZE_PER_IMAGE = 512
cfg.MODEL.ROI_HEADS.NUM_CLASSES = len(classes)
```

As you can see in the preceding code, you can set up all the major hyperparameters needed for training the model. `merge_from_file` imports all the core parameters from a pre-existing configuration file that was used for pre-training `mask_rcnn` with `FPN` as the backbone. This will also contain additional information on the pre-training experiment, such as the optimizer and loss functions. The hyperparameters that have been set, for our purpose, in `cfg` are self-explanatory.

5. Train the model:

```
os.makedirs(cfg.OUTPUT_DIR, exist_ok=True)
trainer = DefaultTrainer(cfg)
trainer.resume_or_load(resume=False)
trainer.train()
```

With the preceding lines of code, we can train a model to predict classes, bounding boxes, and also the segmentation of objects belonging to the de-

finer classes within our custom dataset.

6. Save the model in a folder:

```
!cp output/model_final.pth output/trained_model.pth
```

By this point, we have trained our model. In the next section, we will make inferences on a new image so that we are able to identify objects in a given image using our pre-trained model.

Making inferences on a new image

To perform inference on a new image, we load the path, set the probability threshold, and pass it through the `DefaultPredictor` method, as follows:

1. Load the weights with the trained model. Use the same `cfg` and load the model weights as shown in the following code:

```
cfg.MODEL.WEIGHTS = os.path.join(cfg.OUTPUT_DIR, "trained_model.pth")
```

2. Set the threshold for the probability of the object belonging to a certain class:

```
cfg.MODEL.ROI_HEADS.SCORE_THRESH_TEST = 0.25
```

3. Define the `predictor` method:

```
predictor = DefaultPredictor(cfg)
```

4. Perform segmentation on the image of interest and visualize it. In the following code, we are randomly plotting 30 training images (note that we haven't created validation data; we have left this as an exercise for you), but you can also load your own image path in place of `choose(files)`:

```
from detectron2.utils.visualizer import ColorMode
files = Glob('train/myData2020')
for _ in range(30):
    im = cv2.imread(choose(files))
    outputs = predictor(im)
    v = Visualizer(im[:, :, ::-1], scale=0.5,
                  metadata=MetadataCatalog.get("dataset_train"),
                  instance_mode=ColorMode.IMAGE_BW
    )
    # remove the colors of unsegmented pixels.
    # This option is only available for segmentation models
    out = v.draw_instance_predictions(outputs["instances"].to("cpu"))
    show(out.get_image())
```

`Visualizer` is Detectron2's way of plotting object instances. Given that the predictions (present in the `outputs` variable) are a mere dictionary

of tensors, `Visualizer` converts them into pixel information and draws them on an image. Let's see what each input means:

1. `im`: The image we want to visualize.
2. `scale`: The size of the image when plotted. Here, we are asking it to shrink the image down to 50%.
3. `metadata`: We need class-level information for the dataset, mainly the index-to-class mapping so that when we send the raw tensors as input to be plotted, the class will decode them into actual human-readable classes.
4. `instance_mode`: We are asking the model to only highlight the segmented pixels.
5. Finally, once the class is created (in our example, it is `v`), we can ask it to draw instance predictions coming from the model and show the image.

The preceding code gives the following output. Notice that we are able to identify the pixels corresponding to the elephants fairly accurately:

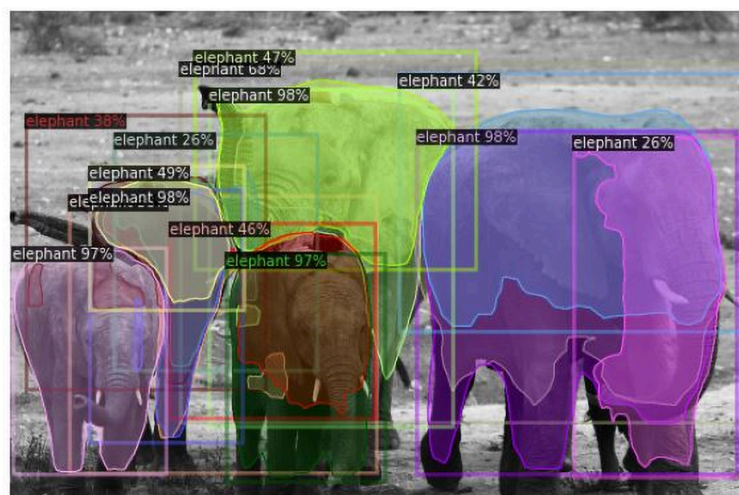


Figure 10.1: Instance segmentation prediction

Now that we have learned about leveraging Detectron2 to identify the pixels corresponding to classes within an image, in the next section, we will learn about leveraging Detectron2 to perform the detection of human poses present in an image.

Human pose detection

In the previous section, we learned about detecting multiple objects and segmenting them. Now we will learn about detecting multiple people in an image, as well as detecting the keypoints of various body parts of the people present in the image using Detectron2. Detecting keypoints comes in handy in multiple use cases, such as in sports analytics and security. For this exercise, we will be leveraging the pre-trained keypoint model that is available in the configuration file:



The following code can be found in the `Human_pose_detection.ipynb` file located in the `Chapter10` folder on GitHub at <https://bit.ly/mcvp-2e>.

1. Install all the requirements as shown in the previous section:

```
%cd /content/
# install detectron2:
!git clone https://github.com/facebookresearch/detectron2
%cd /content/detectron2
%pip install -r requirements.txt
!python setup.py install
%pip install git+https://github.com/facebookresearch/fvcore.git
%cd /content/
%pip install torch_snippets
%pip install pyyaml==5.1 pycocotools>=2.0.1
from torch_snippets import *
import detectron2
from detectron2.utils.logger import setup_logger
setup_logger()
from detectron2 import model_zoo
from detectron2.engine import DefaultPredictor
from detectron2.config import get_cfg
from detectron2.utils.visualizer import Visualizer
from detectron2.data import MetadataCatalog, DatasetCatalog
```

2. Fetch the configuration file and load the pre-trained keypoint detection model present in Detectron2:

```
cfg = get_cfg() # get a fresh new config
cfg.merge_from_file(model_zoo.get_config_file("COCO-Keypoints/keypoint_rcnn_R_50_FPN_3x.yaml"))
```

3. Specify the configuration parameters:

```
cfg.MODEL.ROI_HEADS.SCORE_THRESH_TEST = 0.5 # set threshold
# for this model
cfg.MODEL.WEIGHTS = model_zoo.get_checkpoint_url("COCO-Keypoints/keypoint_rcnn_R_50_FPN_3x.yaml")
predictor = DefaultPredictor(cfg)
```

4. Load the image that we want to predict:

```
from torch_snippets import read, resize
!wget -q https://i.imgur.com/ldzGSHk.jpg -O image.png
im = read('image.png', 1)
im = resize(im, 0.5) # resize image to half its dimensions
```

5. Predict on the image and plot the keypoints:

```
outputs = predictor(im)
v = Visualizer(im[:, :, :-1],
               MetadataCatalog.get(cfg.DATASETS.TRAIN[0]),
               scale=1.2)
out = v.draw_instance_predictions(outputs["instances"].to("cpu"))
import matplotlib.pyplot as plt
```

```
%matplotlib inline
plt.imshow(out.get_image())
```

The preceding code gives output as follows. We can see that the model is able to identify the various body pose keypoints corresponding to the people in the image accurately:



Figure 10.2: (Left) Input image (Right) Predicted keypoints overlaid on original image

In this section, we have learned how to perform keypoint detection using the Detectron2 platform. In the next section, we will learn about implementing a modified VGG architecture from scratch to estimate the number of people present in an image.

Crowd counting

Imagine a scenario where you are given a picture of a crowd and are asked to estimate the number of people present in the image. A crowd counting model comes in handy in such a scenario. Before we go ahead and build a model to perform crowd counting, let's understand the data available and the model architecture first.

In order to train a model that predicts the number of people in an image, we will have to load the images first. The images should constitute the location of the center of the heads of all the people present in the image. A sample of the input image and the location of the center of the heads of the respective people in the image is as follows:

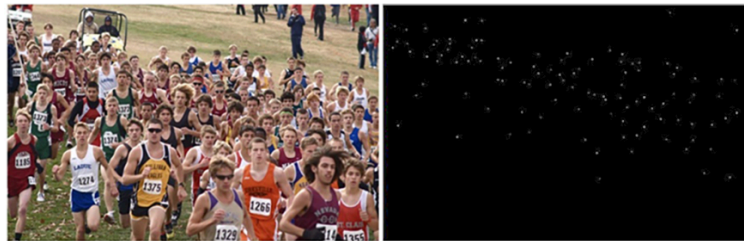


Figure 10.3: (Left) Original image (Right) Plot of the center of the heads of people in the image

Source: ShanghaiTech dataset licensed under BSD 2-Clause "Simplified" License (<https://github.com/desenzhou/ShanghaiTechDataset>)

In the preceding example, the image representing ground truth (on the right – the center of the heads of the people present in the image) is extremely sparse. There are exactly N white pixels, where N is the number of people in the image. Let's zoom in to the top-left corner of the image and see the same map again:

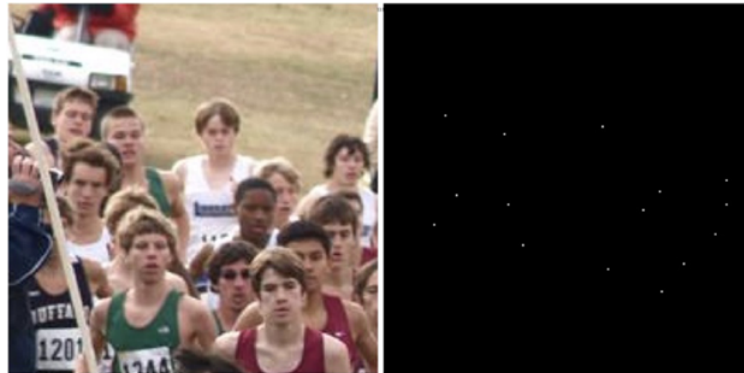


Figure 10.4: Zoomed in version of Figure 10.3

In the next step, we transform the ground truth sparse image into a density map that represents the number of people in that region of the image:

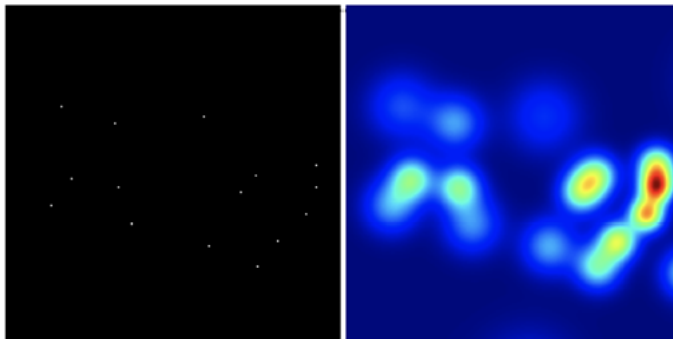


Figure 10.5: (Left) Plot of centers of heads in image (Right) Density map

The final input-output pair of the same crop would look like this:

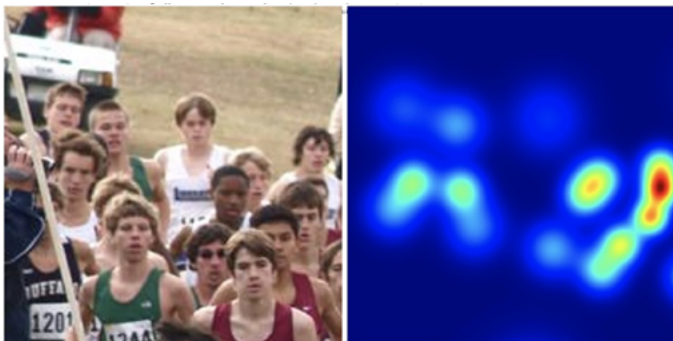


Figure 10.6: (Left) Input (Right) Output of the zoomed in image

The final input-output pair for the entire image would look like this:



Figure 10.7: (Left) Original input image (Right) Density map

Note that, in the preceding image, when two people are close to each other, the pixel intensity is high. However, when a person is far away from the rest, the pixel density corresponding to the person is more evenly spread out, resulting in a lower pixel intensity corresponding to the person who is far away from the rest. Essentially, the heatmap is generated in a way that the sum of the pixel values is equal to the number of people present in the image.

Now that we are in a position to accept an input image and the location of the center of the heads of the people in the image (which is processed to fetch the ground truth output heatmap), we will leverage the architecture detailed in the paper titled *CSRNet: Dilated Convolutional Neural Networks for Understanding the Highly Congested Scenes* (<https://arxiv.org/pdf/1802.10062.pdf>) to predict the number of people present in an image. The model architecture is as follows:

Configurations of CSRNet			
A	B	C	D
input(unfixed-resolution color image)			
front-end (fine-tuned from VGG-16)			
conv3-64-1			
conv3-64-1			
max-pooling			
conv3-128-1			
conv3-128-1			
max-pooling			
conv3-256-1			
conv3-256-1			
conv3-256-1			
max-pooling			
conv3-512-1			
conv3-512-1			
conv3-512-1			
back-end (four different configurations)			
conv3-512-1	conv3-512-2	conv3-512-2	conv3-512-4
conv3-512-1	conv3-512-2	conv3-512-2	conv3-512-4
conv3-512-1	conv3-512-2	conv3-512-2	conv3-512-4
conv3-256-1	conv3-256-2	conv3-256-4	conv3-256-4
conv3-128-1	conv3-128-2	conv3-128-4	conv3-128-4
conv3-64-1	conv3-64-2	conv3-64-4	conv3-64-4
conv1-1-1			

Figure 10.8: CSRNet architecture

Notice in the preceding structure of the model architecture that we are passing the image through four additional layers of convolutions after first passing it through the standard VGG-16 backbone. This output is passed through one of the four configurations and finally through a $1 \times 1 \times 1$ convolution layer. We will be using the A configuration as it is the smallest.

Next, we perform **Mean Squared Error (MSE)** loss minimization on the output image to arrive at the optimal weight values while keeping track of the actual crowd count using MAE. One additional detail of the architecture is that the authors used **dilated convolution** instead of normal convolution. A typical dilated convolution looks as follows:

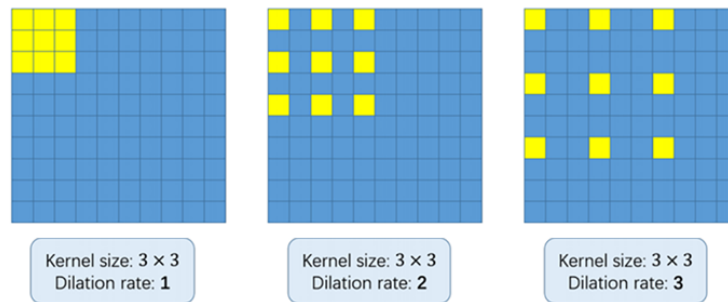


Figure 10.9: Examples of dilated kernels

Source: *arXiv:1802.10062 [cs.CV]*

The preceding diagram on the left represents a typical kernel that we have been working on so far. The diagrams in the middle and on the right represent the dilated kernels, which have a gap between individual pixels. This way, the kernel has a larger receptive field, which can come in handy as we need to understand the number of people near a given person in order to estimate the pixel density corresponding to the person. We are using a dilated kernel (of nine parameters) instead of a normal kernel (which will have 49 parameters to be equivalent to a dilation rate of three kernels) to capture more information with fewer parameters.

With an understanding of how the model is to be architected in place, let's go ahead and code the model to perform crowd counting next.



For those of you looking to understand the working details, we suggest you go through the paper here: <https://arxiv.org/pdf/1802.10062.pdf>. The model we will be training in the following section is inspired by this paper.

Implementing crowd counting

The strategy that we'll adopt to perform crowd counting is as follows:

1. Import the relevant packages and dataset.

2. The dataset that we will be working on – the ShanghaiTech dataset – already has the center of faces converted into a distribution based on Gaussian filter density, so we need not perform it again. Map the input image and the output Gaussian density map using a network.
3. Define a function to perform dilated convolution.
4. Define the network model and train on batches of data to minimize the MSE.

Let's go ahead and code up our strategy as follows:



The following code can be found in the crowd_counting.ipynb file located in the Chapter10 folder on GitHub at <https://bit.ly/mcnp-2e>.

1. Import the packages and download the dataset:

```
%writefile kaggle.json
{"username": "xx", "key": "xx"}
!mkdir -p ~/.kaggle
!mv kaggle.json ~/.kaggle/
!ls ~/.kaggle
!chmod 600 /root/.kaggle/kaggle.json
%%time
%cd /content
import os
if not os.path.exists('shanghaitech-with-people-density-map'):
    print('downloading data...')
    !kaggle datasets download -d tthien/shanghaitech-with-people-density-map/
    print('unzipping data...')
    !unzip -qq shanghaitech-with-people-density-map.zip
if not os.path.exists('CSRNet-pytorch/'):
    %pip install -U scipy torch_snippets torch_summary
    !git clone https://github.com/sizhky/CSRNet-pytorch.git
%cd CSRNet-pytorch
!ln -s ../shanghaitech_with_people_density_map
from torch_snippets import *
import h5py
from scipy import io
```

2. Provide the location of the images (image_folder), the ground truth (gt_folder), and the heatmap folders (heatmap_folder):

```
part_A = Glob('shanghaitech_with_people_density_map/\
ShanghaiTech/part_A/train_data/');
image_folder = 'shanghaitech_with_people_density_map/\
ShanghaiTech/part_A/train_data/images/'
heatmap_folder = 'shanghaitech_with_people_density_map/\
ShanghaiTech/part_A/train_data/ground-truth-h5/'
gt_folder = 'shanghaitech_with_people_density_map/\
ShanghaiTech/part_A/train_data/ground-truth/'
```

3. Define the training and validation datasets and dataloaders:

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
tfm = T.Compose([
```

```

        T.ToTensor())
    ])
class Crowds(Dataset):
    def __init__(self, stems):
        self.stems = stems
    def __len__(self):
        return len(self.stems)
    def __getitem__(self, ix):
        _stem = self.stems[ix]
        image_path = f'{image_folder}/{_stem}.jpg'
        heatmap_path = f'{heatmap_folder}/{_stem}.h5'
        gt_path = f'{gt_folder}/GT_{_stem}.mat'
        pts = io.loadmat(gt_path)
        pts = len(pts['image_info'])[0,0][0,0][0])
        image = read(image_path, 1)
        with h5py.File(heatmap_path, 'r') as hf:
            gt = hf['density'][:, :]
            gt = resize(gt, 1/8)*64
        return image.copy(), gt.copy(), pts
    def collate_fn(self, batch):
        ims, gts, pts = list(zip(*batch))
        ims = torch.cat([tfm(im)[None] for im in ims]).to(device)
        gts = torch.cat([tfm(gt)[None] for gt in gts]).to(device)
        return ims, gts, torch.tensor(pts).to(device)
    def choose(self):
        return self[randint(len(self))]
from sklearn.model_selection import train_test_split
trn_stems, val_stems = train_test_split(stems(Glob(image_folder)),
                                       random_state=10)

trn_ds = Crowds(trn_stems)
val_ds = Crowds(val_stems)
trn_dl = DataLoader(trn_ds, batch_size=1, shuffle=True,
                   collate_fn=trn_ds.collate_fn)
val_dl = DataLoader(val_ds, batch_size=1, shuffle=True,
                   collate_fn=val_ds.collate_fn)

```

Note that the only addition to the typical dataset `class` that we have written so far is the lines of code in bold in the preceding code. We are re-sizing the ground truth as the output of our network would be shrunk to 1/8th of the original size, and hence we are multiplying the `map` by `64` so that the `sum` of the image pixels will be scaled back to the original crowd count.

4. Define the network architecture by implementing the following steps:

1. Define the function that enables dilated convolutions

(`make_layers`):

```

import torch.nn as nn
import torch
from torchvision import models
from utils import save_net, load_net
def make_layers(cfg, in_channels = 3, batch_norm=False, dilation = False):
    if dilation:
        d_rate = 2
    else:
        d_rate = 1
    layers = []
    for v in cfg:

```

```

if v == 'M': # M is for maxpooling
    layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
else:
    conv2d = nn.Conv2d(in_channels=v, kernel_size=3,
                       padding=d_rate, dilation=d_rate)
    if batch_norm:
        layers += [conv2d, nn.BatchNorm2d(v),
                   nn.ReLU(inplace=True)]
    else:
        layers += [conv2d, nn.ReLU(inplace=True)]
    in_channels = v
return nn.Sequential(*layers)

```

2. Define the network architecture – CSRNet :

```

class CSRNet(nn.Module):
    def __init__(self, load_weights=False):
        super(CSRNet, self).__init__()
        self.seen = 0
        self.frontend_feat = [64, 64, 'M', 128, 128,
                              'M', 256, 256, 256, 'M', 512, 512, 512]
        self.backend_feat = [512, 512, 512, 256, 128, 64]
        self.frontend = make_layers(self.frontend_feat)
        self.backend = make_layers(self.backend_feat, in_channels = 512,
                                   dilation = True)
        self.output_layer = nn.Conv2d(64, 1, kernel_size=1)
        if not load_weights:
            mod = models.vgg16(pretrained = True)
            self._initialize_weights()
            items = list(self.frontend.state_dict().items())
            _items = list(mod.state_dict().items())
            for i in range(len(self.frontend.state_dict().items())):
                items[i][1].data[:] = _items[i][1].data[:]
        def forward(self, x):
            x = self.frontend(x)
            x = self.backend(x)
            x = self.output_layer(x)
            return x
        def _initialize_weights(self):
            for m in self.modules():
                if isinstance(m, nn.Conv2d):
                    nn.init.normal_(m.weight, std=0.01)
                    if m.bias is not None:
                        nn.init.constant_(m.bias, 0)
                elif isinstance(m, nn.BatchNorm2d):
                    nn.init.constant_(m.weight, 1)
                    nn.init.constant_(m.bias, 0)

```

5. Define the functions to train and validate a batch of data:

```

def train_batch(model, data, optimizer, criterion):
    model.train()
    optimizer.zero_grad()
    ims, gts, pts = data
    _gts = model(ims)
    loss = criterion(_gts, gts)
    loss.backward()
    optimizer.step()
    pts_loss = nn.L1Loss()(_gts.sum(), gts.sum())

```

```

        return loss.item(), pts_loss.item()
    @torch.no_grad()
    def validate_batch(model, data, criterion):
        model.eval()
        ims, gts, pts = data
        _gts = model(ims)
        loss = criterion(_gts, gts)
        pts_loss = nn.L1Loss()(_gts.sum(), gts.sum())
        return loss.item(), pts_loss.item()

```

6. Train the model over increasing epochs:

```

model = CSRNet().to(device)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-6)
n_epochs = 20
log = Report(n_epochs)
for ex in range(n_epochs):
    N = len(trn_dl)
    for bx, data in enumerate(trn_dl):
        loss, pts_loss = train_batch(model, data, optimizer, criterion)
        log.record(ex+(bx+1)/N, trn_loss=loss,
                    trn_pts_loss=pts_loss, end='\r')
    N = len(val_dl)
    for bx, data in enumerate(val_dl):
        loss, pts_loss = validate_batch(model, data, criterion)
        log.record(ex+(bx+1)/N, val_loss=loss,
                    val_pts_loss=pts_loss, end='\r')
    log.report_avgs(ex+1)
    if ex == 10: optimizer = optim.Adam(model.parameters(),
                                         lr=1e-7)

```

The preceding code results in a variation in the training and validation loss (here, the loss is the MAE of the crowd count), as follows:

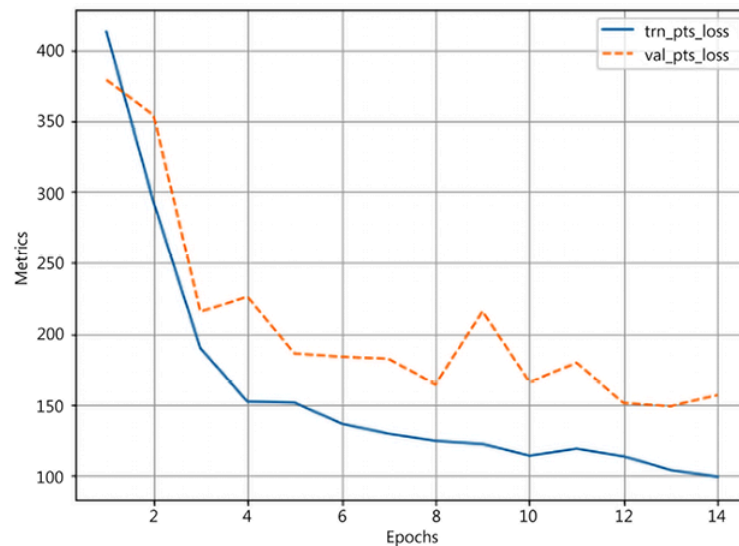


Figure 10.10: Training and validation loss over increasing epochs

From the preceding plot, we can see that we are off in our predictions by around 150 people. We can improve the model in the following two ways:

1. By using data augmentation and training on crops of the original image
 2. By using a larger network (we used the *A* configuration, while *B*, *C*, and *D* are larger)
7. Make inferences on a new image by fetching a test image and normalizing it:

```
from matplotlib import cm as c
from torchvision import datasets, transforms
from PIL import Image
transform=transforms.Compose([
    transforms.ToTensor(),transforms.Normalize(\
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]),
])
test_folder = 'shanghaitech_with_people_density_map/\
ShanghaiTech/part_A/test_data/'
imgs = Glob(f'{test_folder}/images')
f = choose(imgs)
print(f)
img = transform(Image.open(f).convert('RGB')).to(device)
```

Then pass the image through the trained model:

```
output = model(img[None])
print("Predicted Count : ", int(output.detach().cpu().sum().numpy()))
temp = np.asarray(output.detach().cpu()\
    .reshape(output.detach().cpu()\
    .shape[2],output.detach()\
    .cpu().shape[3]))
plt.imshow(temp,cmap = c.jet)
plt.show()
```

The preceding code results in a heatmap (right image) of the input image (left image). We can see that the model predicted the heatmap reasonably accurately and the prediction count of people is close to the actual value:

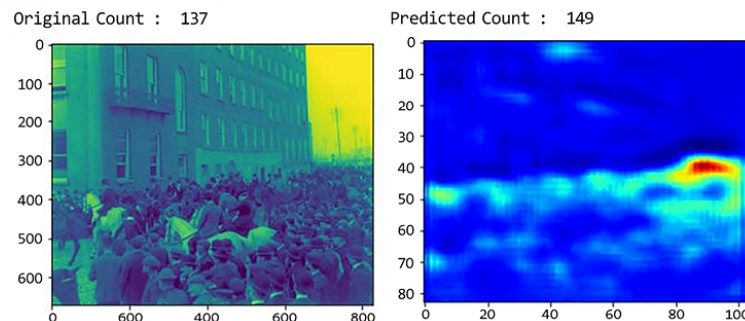


Figure 10.11: (Left) Input image (Right) Predicted density map and the count of people

In the next section, we will continue to work on additional applications and will leverage a U-Net architecture to colorize an image.

Image colorization

Imagine a scenario where you are given a bunch of black-and-white images and are asked to turn them into color images. How would you solve this problem? One way to solve this is by using a pseudo-supervised pipeline where we take a raw image, convert it into black and white images, and treat them as input-output pairs.

We will demonstrate this by leveraging the CIFAR-10 dataset to perform colorization on images. The strategy that we will adopt as we code up the image colorization network is as follows:

1. Take the original color image in the training dataset and convert it into grayscale to fetch the input (grayscale) and output (original colored image) combination.
2. Normalize the input and output.
3. Build a U-Net architecture.
4. Train the model over increasing epochs.

With the preceding strategy in place, let's go ahead and code up the model as follows:



The following code can be found in the `Image colorization.ipynb` file located in the `Chapter10` folder on GitHub at <https://bit.ly/mcvcv-2e>.

1. Install the required packages and import them:

```
!pip install torch_snippets
from torch_snippets import *
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

2. Download the dataset and define the training and validation datasets and dataloaders:

1. Download the dataset:

```
from torchvision import datasets
import torch
data_folder = '~/cifar10/cifar/'
datasets.CIFAR10(data_folder, download=True)
```

2. Define the training and validation datasets and dataloaders:

```
class Colorize(torchvision.datasets.CIFAR10):
    def __init__(self, root, train):
        super().__init__(root, train)

    def __getitem__(self, ix):
        im, _ = super().__getitem__(ix)
        bw = im.convert('L').convert('RGB')
        bw, im = np.array(bw)/255., np.array(im)/255.
        bw, im = [torch.tensor(i).permute(2,0,1)\
                  .to(device).float() for i in [bw,im]]
```

```

        return bw, im
    trn_ds = Colorize('~/.cifar10/cifar/', train=True)
    val_ds = Colorize('~/.cifar10/cifar/', train=False)
    trn_dl = DataLoader(trn_ds, batch_size=256, shuffle=True)
    val_dl = DataLoader(val_ds, batch_size=256, shuffle=False)

```

3. A sample of the input and output images is as follows:

```

a,b = trn_ds[0]
subplots([a,b], nc=2)

```

The preceding code results in the following output:



Figure 10.12: (Left) Input image (Right) Colorized image

Note that CIFAR-10 has images that are 32 x 32 in shape.

3. Define the network architecture:

```

class Identity(nn.Module):
    def __init__(self):
        super().__init__()
    def forward(self, x):
        return x

class DownConv(nn.Module):
    def __init__(self, ni, no, maxpool=True):
        super().__init__()
        self.model = nn.Sequential(
            nn.MaxPool2d(2) if maxpool else Identity(),
            nn.Conv2d(ni, no, 3, padding=1),
            nn.BatchNorm2d(no),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(no, no, 3, padding=1),
            nn.BatchNorm2d(no),
            nn.LeakyReLU(0.2, inplace=True),
        )
    def forward(self, x):
        return self.model(x)

class UpConv(nn.Module):
    def __init__(self, ni, no, maxpool=True):
        super().__init__()
        self.convtranspose = nn.ConvTranspose2d(ni, no, 2, stride=2)
        self.convlayers = nn.Sequential(
            nn.Conv2d(no+no, no, 3, padding=1),
            nn.BatchNorm2d(no),
            nn.LeakyReLU(0.2, inplace=True),

```

```

        nn.Conv2d(no, no, 3, padding=1),
        nn.BatchNorm2d(no),
        nn.LeakyReLU(0.2, inplace=True),
    )

    def forward(self, x, y):
        x = self.convtranspose(x)
        x = torch.cat([x,y], axis=1)
        x = self.convlayers(x)
        return x

class UNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.d1 = DownConv( 3, 64, maxpool=False)
        self.d2 = DownConv( 64, 128)
        self.d3 = DownConv( 128, 256)
        self.d4 = DownConv( 256, 512)
        self.d5 = DownConv( 512, 1024)
        self.u5 = UpConv (1024, 512)
        self.u4 = UpConv ( 512, 256)
        self.u3 = UpConv ( 256, 128)
        self.u2 = UpConv ( 128, 64)
        self.u1 = nn.Conv2d(64, 3, kernel_size=1, stride=1)

    def forward(self, x):
        x0 = self.d1( x) # 32
        x1 = self.d2(x0) # 16
        x2 = self.d3(x1) # 8
        x3 = self.d4(x2) # 4
        x4 = self.d5(x3) # 2
        X4 = self.u5(x4, x3)# 4
        X3 = self.u4(X4, x2)# 8
        X2 = self.u3(X3, x1)# 16
        X1 = self.u2(X2, x0)# 32
        X0 = self.u1(X1) # 3
        return X0

```

4. Define the model, optimizer, and loss function:

```

def get_model():
    model = UNet().to(device)
    optimizer = optim.Adam(model.parameters(), lr=1e-3)
    loss_fn = nn.MSELoss()
    return model, optimizer, loss_fn

```

5. Define the functions to train and validate a batch of data:

```

def train_batch(model, data, optimizer, criterion):
    model.train()
    x, y = data
    _y = model(x)
    optimizer.zero_grad()
    loss = criterion(_y, y)
    loss.backward()
    optimizer.step()
    return loss.item()

@torch.no_grad()
def validate_batch(model, data, criterion):
    model.eval()
    x, y = data

```

```
_y = model(x)
loss = criterion(_y, y)
return loss.item()
```

6. Train the model over increasing epochs:

```
model, optimizer, criterion = get_model()
exp_lr_scheduler = optim.lr_scheduler.StepLR(optimizer,
                                              step_size=10, gamma=0.1)
_val_dl = DataLoader(val_ds, batch_size=1, shuffle=True)
n_epochs = 100
log = Report(n_epochs)
for ex in range(n_epochs):
    N = len(trn_dl)
    for bx, data in enumerate(trn_dl):
        loss = train_batch(model, data, optimizer, criterion)
        log.record(ex+(bx+1)/N, trn_loss=loss, end='\r')
        if (bx+1)%50 == 0:
            for _ in range(5):
                a,b = next(iter(_val_dl))
                _b = model(a)
                subplots([a[0], b[0], _b[0]], nc=3, figsize=(5,5))
    N = len(val_dl)
    for bx, data in enumerate(val_dl):
        loss = validate_batch(model, data, criterion)
        log.record(ex+(bx+1)/N, val_loss=loss, end='\r')

    exp_lr_scheduler.step()
    if (ex+1) % 5 == 0: log.report_avgs(ex+1)
    for _ in range(5):
        a,b = next(iter(_val_dl))
        _b = model(a)
        subplots([a[0], b[0], _b[0]], nc=3, figsize=(5,5))
log.plot_epochs()
```

The preceding code generates an output, as follows. We can see that the model is able to color the grayscale image reasonably well:

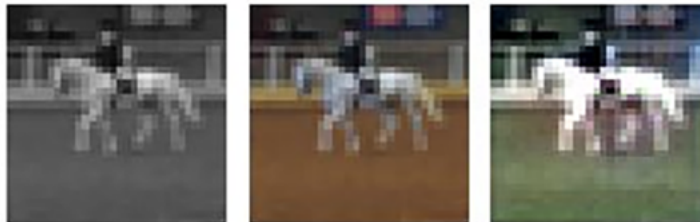


Figure 10.13: (Left) Input image (Middle) Original image (Right) Predicted image

So far, we have learned about leveraging Detectron2 for segmentation and keypoint detection, dilated convolutions in crowd counting, and U-Net in image colorization. In the next section, we will learn about leveraging YOLO for 3D object detection.

3D object detection with point clouds

We now know how to predict a bounding rectangle on 2D images using algorithms that have the core underlying concept of anchor boxes. Let's learn how the same concept can be extended to predict 3D bounding boxes around objects.

In a self-driving car, tasks such as pedestrian/obstacle detection and route planning cannot happen without knowing the environment. Predicting 3D object locations along with their orientations is an important task. Not only is the 2D bounding box around obstacles important but knowing the distance from the object, height, width, and orientation of the obstacle are also critical to navigating safely in the 3D world.

In this section, we will learn how YOLO is used to predict the 3D orientation and position of cars and pedestrians on a real-world dataset.



The instructions for downloading the data, training, and testing sets are all given in this GitHub repo:

<https://github.com/sizhky/Complex-YOLOv4-Pytorch/blob/master/README.md#training-instructions>.

Given that there are very few openly available 3D datasets, we have chosen the most-used dataset for this exercise, which you still need to register for to download. We have provided the instructions for registration at the preceding link as well.

Theory

One of the well-known sensors for collecting real-time 3D data is **Light Detection and Ranging (LIDAR)**. It is a laser mounted on a rotating apparatus that fires beams of lasers hundreds of times every second. Another sensor receives the reflection of the laser from surrounding objects and calculates how far the laser has traveled before encountering an obstruction. Doing this in all directions of a car will result in a 3D point cloud of distances that is reflective of the environment itself. In the dataset that we will learn about, we have obtained the 3D point clouds from specific hardware developed by Velodyne. Let's understand how the input and output are encoded for 3D object detection.

Input encoding

Our raw inputs are going to be 3D point clouds presented to us in the form of `.bin` files. Each can be loaded as a NumPy array using `np.fromfile(<filepath>)` and here's how the data looks for a sample file:

```
files = Glob('training/velodyne')
F = choose(files)
pts = np.fromfile(F, dtype=np.float32).reshape(-1, 4)
pts
```



These files are found in the `dataset/.../training/velodyne` directory after download-



ing and moving the raw files as per the GitHub repo instructions.

The preceding code gives the following output:

```
array([[62.502,  8.628,  2.343,  0.  ],
       [62.468,  8.824,  2.342,  0.  ],
       [66.793, 10.832,  2.497,  0.  ],
       ...,
       [ 3.75 , -1.418, -1.753,  0.25 ],
       [ 3.759, -1.409, -1.756,  0.32 ],
       [ 3.767, -1.398, -1.758,  0.  ]], dtype=float32)
```

Figure 10.14: Input array

This can be visualized as follows:

```
# take the points and remove faraway points
x,y,z = np.clip(pts[:,0], 0, 50),
        np.clip(pts[:,1], -25, 25),
        np.clip(pts[:,2], -3, 1.27)
fig = go.Figure(data=[go.Scatter3d(\
    x=x, y=y, z=z, mode='markers',
    marker=dict(
        size=2,
        color=z, # set color to a list of desired values
        colorscale='Viridis', # choose a colorscale
        opacity=0.8
    )
)])
fig.update_layout(margin=dict(l=0, r=0, b=0, t=0))
fig.show()
```

The preceding code results in the following output:

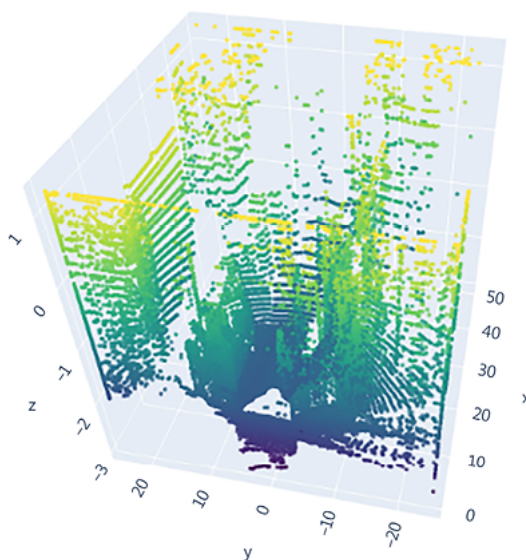


Figure 10.15: Visualization of input array

We can convert this information into an image of a bird's-eye view by performing the following steps:

1. Project the 3D point cloud onto the XY plane (ground) and split it into a grid with a resolution of 8 cm^2 per grid cell.
2. For each cell, compute the following and associate them with the specified channel:
 1. Red channel: The height of the highest point in the grid
 2. Green channel: The intensity of the highest point in the grid
 3. Blue channel: The number of points in the grid divided by 64 (which is a normalizing factor)

For example, the reconstructed top view of the cloud may look like this:

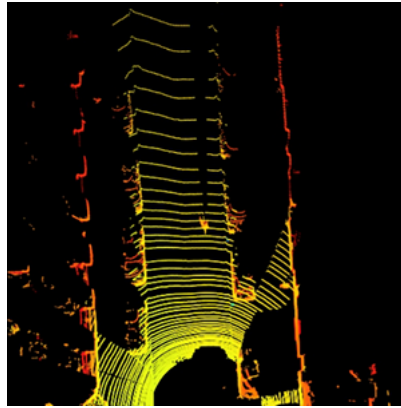


Figure 10.16: Bird's eye view of input image

You can clearly see the “shadows” in the image, indicating that there is an obstacle. This is how we create an image from the LIDAR point cloud data.



We have taken 3D point clouds as the raw input and obtained the bird's-eye image as the output. This is the preprocessing step necessary to create the image that is going to be the input for the YOLO model.

Output encoding

Now that we have the bird's-eye image (of the 3D point cloud) as input to the model, the model needs to predict the following real-world features:

- What the object (**class**) present in the image is
- How far the object is (in meters) from the car on the east-west axis (**x**)
- How far the object is (in meters) from the car on the north-south axis (**y**)
- What the orientation of the object (**yaw**) is
- How big the object is (the **length** and **width** of the object in meters)

It is possible to predict the bounding box in the pixel coordinate system (of the bird's-eye image), but it does not have any real-world significance as the predictions would still be in pixel space (in a bird's-eye view). In

this case, we need to convert these pixel coordinate (of the bird's-eye view) bounding box predictions into real-world coordinates in meters. To avoid additional steps during postprocessing, we are directly predicting the real-world values.

Furthermore, in a realistic scenario, the object could be oriented in any direction. If we only calculate the length and width, it will not be sufficient to describe the tight bounding box. An example of such a scenario is as follows:

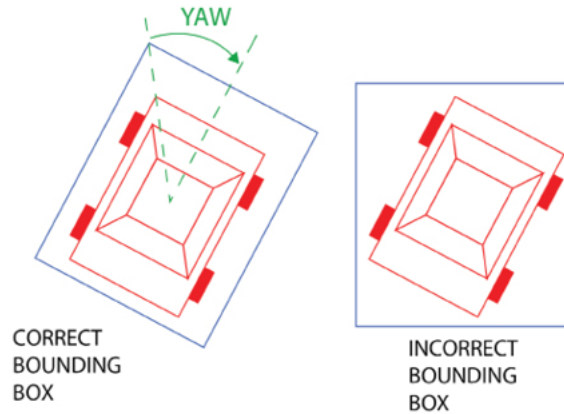


Figure 10.17: Bounding box representation

To get a tight bounding box for the object, we also need the information on which direction the obstacle is facing, and hence we also need the additional yaw parameter. Formally, it is the orientation of the object with the north-south axis.

First, the YOLO model uses an anchor grid of 32 x 64 cells (more width than height), taking into consideration that the car's dashcam (and hence LIDAR) views are wider than they are tall. The model uses two losses for the task. The first one is the normal YOLO loss (which is responsible for predicting the x , y , l , w and class) we learned about in *Chapter 8, Advanced Object Detection*, and another loss called the Euler loss, which exclusively predicts the yaw. Formally, the set of equations to predict the final bounding boxes from the model's outputs is as follows:

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

$$b_w = p_w e^{t_w}$$

$$b_l = p_l e^{t_l}$$

$$b_\phi = \arctan2(t_{lm}, t_{Re})$$

Here, b_x , b_y , b_w , b_l , and b_ϕ are the x and y coordinate values, the width, the length, and the yaw of the obstacle, respectively. t_x , t_y , t_w , t_l , t_{lm} , and t_{Re} are the six regression values that are being predicted from Note that even though there are only 5 values to be predicted, the angle ϕ is being pre-

dicted using two auxiliary values t_{Im} and t_{Re} , which represent imaginary and real targets, respectively. These are just names used by the official implementation and are essentially trying to calculate b_ϕ using the preceding arctan formula. c_x and c_y are the positions of the center of the grid cell within the 32 x 64 matrix and pw and pl are pre-defined priors chosen by taking the average widths and lengths of cars and pedestrians. Furthermore, there are five priors (anchor boxes) in the implementation.



The height of each object of the same class is assumed to be a fixed number.

Refer to the illustration given here, which shows this pictorially:

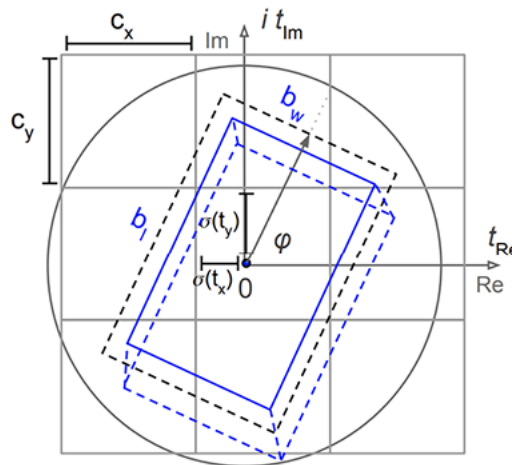


Figure 10.18: Bounding box regression Source: arXiv:1803.06199v2 [cs.CV]

The total loss is calculated as follows:

$$Loss = loss_{YOLO} + loss_{EULER}$$

You already know $Loss_{YOLO}$ from the previous chapter (using t_x , t_y , t_w , and t_l as the targets). Also, note the following:

$$Loss_{EULER} = \sum_{all\ objects} \sum_{all\ grid\ cells} f(object, cell)$$

$$f(object, cell) = \begin{cases} (t_{Im} - \hat{t}_{Im})^2 + (t_{Re} - \hat{t}_{Re})^2 & \text{if object is in cell} \\ 0 & \text{otherwise} \end{cases}$$

Now that we have understood that the fundamentals of 3D object detection are the same as those of 2D object detection (but with more parameters to predict) and the input-output pairs of this task, let's leverage an existing GitHub repo to train our model.



For more details on 3D object detection, refer to the paper Complex-YOLO at <https://arxiv.org/pdf/1803.06199.pdf>.

Training the YOLO model for 3D object detection

The coding effort is largely taken away from the user due to the standardized code. Much like Detectron2, we can train and test the algorithm by ensuring that the data is in the right format in the right location. Once that is ensured, we can train and test the code with a minimal number of lines.

We need to clone the `Complex-YOLOv4-Pytorch` repository first:

```
$ git clone https://github.com/sizhky/Complex-YOLOv4-Pytorch
```

Follow the instructions in the `README.md` file to download and move the datasets to the right locations.



The instructions for downloading the data, training, and testing sets are all given in this GitHub repo:
<https://github.com/sizhky/Complex-YOLOv4-Pytorch/blob/master/README.md#training-instructions>.

Given that there are very few openly available 3D datasets, we have chosen the most-used dataset for this exercise, which you still need to register in order to download. We also give the instructions for registration at the preceding link.

Data format

We can use any 3D point cloud data with ground truths for this exercise. Refer to the `README` file in the GitHub repository for more instructions on how to download and move the data.

The data needs to be stored in the following format in the root directory:

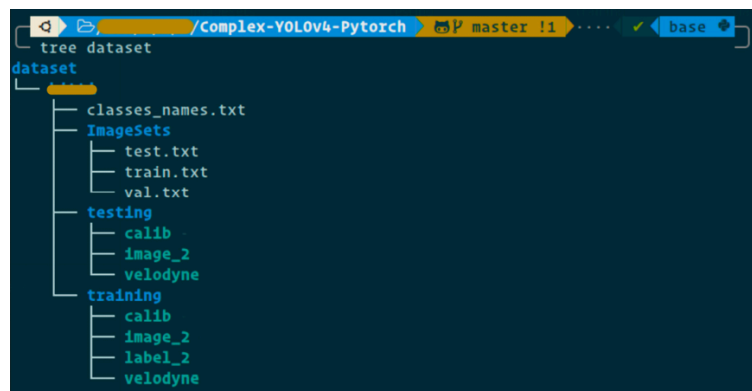


Figure 10.19: Data storage format

The three folders that are new to us are `velodyne`, `calib`, and `label_2`:

- `velodyne` contains a list of `.bin` files that encode 3D point cloud information for corresponding images present in the `image_2` folder.

- `calib` contains calibration files corresponding to each point cloud. The 3D coordinates from the LIDAR point cloud coordinate system can be projected onto the camera coordinate system – that is, the image – by using the 3 x 4 projection matrix present in each file in the `calib` folder. Essentially, the LIDAR sensor captures the points that are slightly offset from what the camera is capturing. This offset is due to the fact that the sensors are mounted a few inches apart from each other. Knowing the right offsets will help us to correctly project bounding boxes and 3D points onto the image from the camera.
- `label_2` contains the ground truths (one ground truth per line) for each image in the form of 15 values that are explained in the following table:

Figure 10.20: Sample ground truth values

Note that our target columns are type (class), *w*, *l*, *x*, *z*, and *ry* (yaw) among the ones seen here. We will ignore the rest of the values for this task.

Data inspection

We can verify that the data is downloaded properly by running the following:

```
$ cd Complex-YOLOv4-Pytorch/src/data_process
$ python kitti_data_loader.py --output-width 600
```

The preceding code shows multiple images, one image at a time. The following is one such example (image source:

<https://arxiv.org/pdf/1803.06199.pdf>):

Figure 10.21: Input image with the corresponding ground truth

Source: [arXiv:1803.06199v2](https://arxiv.org/abs/1803.06199v2) [cs.CV]

Now that we are able to download and view a few images, in the next section, we will learn about training the model to predict 3D bounding boxes.

Training

The training code is wrapped in a single Python file and can be called as follows:

```
$ cd Complex-YOLOv4-Pytorch/src
$ python train.py --gpu_idx 0 --batch_size 2 --num_workers 4 \
  --num_epochs 5
```

The default number of epochs is 300, but the results are fairly reasonable starting at the fifth epoch itself. Each epoch takes 30 to 45 minutes on a GTX 1070 GPU. You can use `--resume_path` to resume training if training cannot be done in a single stretch. The code saves a new checkpoint every five epochs.

Testing

Just like in the preceding *Data inspection* section, the trained model can be tested with the following code:

```
$ cd Complex-YOLOv4-Pytorch/src
$ python test.py --gpu_idx 0 --pretrained_path ../checkpoints/complexer_yolo/Model_complexer_yolo.
```

The main inputs to the code are the checkpoint path and the model configuration path. After giving them and running the code, the following output pops up (image source: <https://arxiv.org/pdf/1803.06199.pdf>):

Figure 10.22: Input image with the corresponding predicted labels and bounding boxes

Source: *arXiv:1803.06199v2 [cs.CV]*

Because of the simplicity of the model, we can use it in real-time scenarios with a normal GPU, getting about 15–20 predictions per second.

So far, we have learnt about scenarios where we take an image/frame as input and predict the class/ object/bounding box. What if we want to recognize an event from a video (or a sequence of frames)? Let's focus on this in the next section.

Action recognition from video

Let's now learn how to use the MMAction toolbox

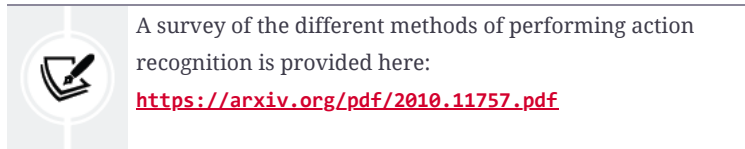
(<https://github.com/open-mmlab/mmaaction>) from the open-mmlab project to perform action recognition. The major features of MMAction are:

- Action recognition on trimmed videos (portion of the video that has an action)
- Temporal action detection (action localization) in untrimmed videos
- Spatial (parts of a frame that indicate an action) and temporal (variation of action across frames) action detection in untrimmed videos
- Support for various action datasets
- Support for multiple action understanding frameworks

First, let us understand how action recognition works. A video is a collection of images that are spaced over time (frames). We have two options of model input – 2D and 3D. 2D model input has a dimension of $F \times CHW$ where F is the number of frames and C , H , W are channels, height, and width respectively. 3D model input has an input dimension of $CFHW$.

In the case of 2D model input, we pass the video (set of frames) through the backbones that we learned about in *Chapter 5* (VGG16, ResNet50) to obtain the intermediate layers. We next pass the intermediate outputs through temporal convolution to aggregate the information of what is happening in each frame. In the case of 3D model input, we pass it through a 3D model backbone like ResNet3D that can inherently process the temporal dimension along with spatial dimensions to fetch the intermediate layers.

Next, we pass the output (across all frames) through a pooling layer (so that the model processes on a reduced dimension that captures the key features) to obtain the penultimate layer, which is then used to predict the different classes.

Figure 10.23: Action recognition workflow

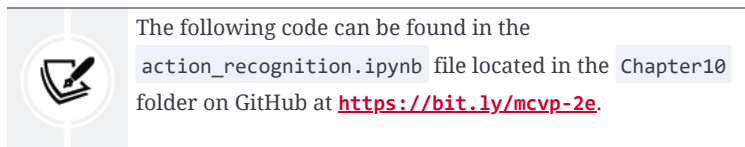
Now that we have an understanding of how action classification is done on a video, let's perform the following:

1. Leverage MMAction out of the box to identify an action in a given video
2. Train MMAction to recognize an action in a custom dataset

Let's get started using MMAction.

Identifying an action in a given video

To identify an action in a given video using MMAction, perform the following steps:



1. Install the dependencies. We'll install `pytorch` (version 2.2.1+u121) and install MIM 0.3.9. MIM provides a unified interface for launching and installing OpenMMLab projects and their extensions and managing the OpenMMLab model zoo. First, install `openmim`, `mmengine`, and `mmcv`. We will then install `mmaction` and its dependencies:

```
%pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
%pip install -U "openmim==0.3.9"
!mim install -U "mmengine==0.10.4"
!mim install "mmcv==2.2.0"
!git clone https://github.com/sizhky/mmaaction2.git -b main
%cd mmaaction2
%pip install -e .
%pip install -r requirements/optional.txt
%pip install "timm==0.9.16"
%pip install "torch-snippets==0.528" lovely-tensors
```

2. Download the pre-trained checkpoints:

```
!mkdir checkpoints
!wget -c https://download.openmmlab.com/mmdetection/v2.0/mmdetection/mmdetection_r50_1x1x3_100e_kinetics400
```

3. Import the required packages:

```
from mmdetection.apis import inference_recognizer, init_recognizer
from mmengine import Config
from torch_snippets import *
from builtins import print
```

4. Initialize the recognizer:

```
# Choose to use a config and initialize the recognizer
config = 'configs/recognition/tsn/tsn_imagenet-pretrained-r50_8xb32-1x1x3-100e_kinetics400-rgb'
config = Config.fromfile(config)
```

Much like `detectron2` and `huggingface`, `mmdetection` is a library that uses config files to create models, dataloaders, and pipelines, as well as trainers. We will continue to leverage it in this and the next section.

```
# Setup a checkpoint file to load
checkpoint = 'checkpoints/tsn_r50_1x1x3_100e_kinetics400_rgb_20200614-e508be42.pth'
# Initialize the recognizer
model = init_recognizer(config, checkpoint, device='cuda:0')
```

The model essentially has two components – a ResNet backbone that takes in a tensor of shape $[F \times 3 \times H \times W]$ and returns a feature vector of shape $[F \times 2048 \times 7 \times 7]$. The head converts this tensor by first averaging each of the 7×7 feature maps. This returns $[F \times 2048 \times 1 \times 1]$. In the next step, the frames are averaged out and the computed tensor will be of shape $[1 \times 2048 \times 1 \times 1]$. This is flattened into $[1 \times 2048]$, which is sent through a linear layer that finally returns the $[1 \times 400]$ tensor where 400 is the number of classes.

5. Use the recognizer to perform inference:

```
from operator import itemgetter
video = 'demo/demo.mp4'
label = 'tools/data/kinetics/label_map_k400.txt'
results = inference_recognizer(model, video)
pred_scores = results.pred_score.cpu().numpy().tolist()
score_tuples = tuple(zip(range(len(pred_scores)), pred_scores))
score_sorted = sorted(score_tuples, key=itemgetter(1), reverse=True)
top5_label = score_sorted[:5]
labels = open(label).readlines()
labels = [x.strip() for x in labels]
results = [(labels[k[0]], k[1]) for k in top5_label]
```

The `inference_recognizer` function is a wrapper around video preprocessing and `model.forward` where the video is loaded as a NumPy array, the frames are resized and reshaped, and the dimensions are set appropriately for the model to accept a $[F \times 3 \times H \times W]$ tensor.

6. Print the predictions:

```
for result in results:
    print(f'{result[0]}: ', result[1])
```

Figure 10.24: Predicted actions

We used the simple trick of treating the frames as batch dimensions. This way, the functionality of ResNet does not change. A new head is used to average out the frames into a single value and the tensor can be simply used to perform classification with cross-entropy loss. As you can see, we used a relatively straightforward path to extend our knowledge of image processing to also perform video classification.

Training a recognizer on a custom dataset

Now that we have learned how to leverage existing architecture for video classification, let's take this a step further and train the same model on a binary classification video dataset of our own. Note that this can be extended to video classification of any number of classes.

To train a new recognizer, we need to perform the following:

1. Let's download a small subset of the dataset present at <https://research.google/pubs/the-kinetics-human-action-video-dataset/>:

```
# download, decompress the data
!rm kinetics400_tiny.zip*
!rm -rf kinetics400_tiny
!wget https://download.openmmlab.com/mmdetection/v2.0/kinetics400_tiny.zip
!unzip kinetics400_tiny.zip > /dev/null
```

The preceding code downloads 40 videos – 30 into the training dataset and 10 into the validation dataset.

The task is binary video classification where there are two classes – “Climbing Rope” (0) and “Blowing Glass” (1)

2. Check the annotation format:

```
!cat kinetics400_tiny/kinetics_tiny_train_video.txt
```

Figure 10.25: Input video path and the corresponding class

Each line above indicates the file path and the label corresponding to it.

3. Modify the config file for training by implementing the following:
 1. Initialize the configuration file:

```
cfg = Config.fromfile('./configs/recognition/tsn/tsn_imagenet-pretrained-r50_8xb32-1x1x3-100e_1
```

As mentioned in the previous section, we will use the config file to create the trainer class (also called the runner).

2. We will modify the configuration values by changing the defaults to fit our use case. The names of the variables should be self-explanatory:

```
from mmengine.runner import set_random_seed
# Modify dataset type and path
cfg.data_root = 'kinetics400_tiny/train/'
cfg.data_root_val = 'kinetics400_tiny/val/'
cfg.ann_file_train = 'kinetics400_tiny/kinetics_tiny_train_video.txt'
cfg.ann_file_val = 'kinetics400_tiny/kinetics_tiny_val_video.txt'
cfg.test_dataloader.dataset.ann_file = 'kinetics400_tiny/kinetics_tiny_val_video.txt'
cfg.test_dataloader.dataset.data_prefix.video = 'kinetics400_tiny/val/'
cfg.train_dataloader.dataset.ann_file = 'kinetics400_tiny/kinetics_tiny_train_video.txt'
cfg.train_dataloader.dataset.data_prefix.video = 'kinetics400_tiny/train/'
cfg.val_dataloader.dataset.ann_file = 'kinetics400_tiny/kinetics_tiny_val_video.txt'
cfg.val_dataloader.dataset.data_prefix.video = 'kinetics400_tiny/val/'
# Modify num classes of the model in cls_head
cfg.model.cls_head.num_classes = 2
# We can use the pre-trained TSN model
cfg.load_from = './checkpoints/tsn_r50_1x1x3_100e_kinetics400_rgb_20200614-e508be42.pth'
# Set up working dir to save files and logs.
cfg.work_dir = './output_dir'
cfg.train_dataloader.batch_size = cfg.train_dataloader.batch_size // 16
cfg.val_dataloader.batch_size = cfg.val_dataloader.batch_size // 16
cfg.optim_wrapper.optimizer.lr = cfg.optim_wrapper.optimizer.lr / 8 / 16
cfg.train_cfg.max_epochs = 10
cfg.train_dataloader.num_workers = 2
cfg.val_dataloader.num_workers = 2
cfg.test_dataloader.num_workers = 2
```

4. Finally, we'll create a runner class and use it to train the recognizer:

```
import os.path as osp
import mmengine
from mmengine.runner import Runner
# Create work_dir
mmengine.mkdir_or_exist(osp.abspath(cfg.work_dir))
# build the runner from config
runner = Runner.from_cfg(cfg)
# start training
runner.train()
```

The preceding code results in a training accuracy of 100%.

Note

Classes like `Runner` in `mmaction`; `Trainer` in `huggingface`, `pytorch lightning`, `pytorch-ignite`, `tensorflow`, and `detectron2`; and `Learner` in `fastai` are all wrappers for the core training components.

- `optimizer.zero_grad()`
- `model.train()`

```
• pred = model(inputs)
• loss = loss_fn(pred, target)
• loss.backward()
• optimizer.step()
```

The differences in each of the libraries' functionalities are merely in the way they were implemented. Their functionalities are almost identical and exploring them is a good exercise for you to understand how to write good deep learning code.

5. Finally, we test the recognizer:

```
runner.test()
```

The preceding results in an accuracy of 90% in Top1-accuracy and 100% in Top5-accuracy.

Summary

In this chapter, we learned about the various practical aspects of dealing with object localization and segmentation. Specifically, we learned about how the Detectron2 platform is leveraged to perform image segmentation and detection, and keypoint detection. In addition, we also learned about some of the intricacies involved in working with large datasets when we were working on fetching images from the Open Images dataset. Next, we worked on leveraging the VGG and U-Net architectures for crowd counting and image colorization, respectively. Then, we understood the theory and implementation steps behind 3D object detection using point cloud images. Finally, we understood ways of performing classification exercises on a sequence of frames (video). As you can see from all these examples, the underlying basics are the same as those described in the previous chapters, with modifications only in the input/output of the networks to accommodate the task at hand.

In the next chapter, we will switch gears and learn about image encoding, which helps in identifying similar images as well as generating new images.

Questions

1. Why is it important to convert datasets into a specific format for Detectron2?
2. It is hard to directly perform a regression of the number of people in an image. What is the key insight that allowed the VGG architecture to perform crowd counting?
3. Explain self-supervision in the case of image-colorization.
4. How did we convert a 3D point cloud into an image that is compatible with YOLO?
5. What is a simple way to handle videos using architectures that work only with images?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/modcv>

